Worksheet No. - 02

Aim/Overview of the practical (A). Write a test script using unit testing framework for addition, subtraction, multiplication and division of two numbers.

Objective: The objective of this experiment is to develop and execute a test script using a unit testing framework to verify the correctness of basic arithmetic operations such as addition, subtraction, multiplication, and division of two numbers. The testing ensures that each function produces the expected output and helps identify errors or unexpected behavior in the program.

Procedure/Algorithm/Code

```
import unittest


def add(a, b):
    return a + b


def subtract(a, b):
    return a - b


def multiply(a, b):
    return a * b


def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b


class TestCalculator(unittest.TestCase):


    results = []


    def test_addition(self):
        expected = 79
```

```
        actual = add(50, 29)
        self.results.append(["Addition", expected, actual,
                             "PASS" if expected == actual else "FAIL"])
        self.assertEqual(expected, actual)

    def test_subtraction(self):
        expected = 30
        actual = subtract(37, 7)
        self.results.append(["Subtraction", expected, actual,
                             "PASS" if expected == actual else "FAIL"])
        self.assertEqual(expected, actual)

    def test_multiplication(self):
        expected = 343
        actual = multiply(7, 49)
        self.results.append(["Multiplication", expected, actual,
                             "PASS" if expected == actual else "FAIL"])
        self.assertEqual(expected, actual)

    def test_division(self):
        expected = 30
        actual = divide(90, 3)
        self.results.append(["Division", expected, actual,
                             "PASS" if expected == actual else "FAIL"])
        self.assertEqual(expected, actual)

if __name__ == "__main__":
    suite = unittest.TestLoader().loadTestsFromTestCase(TestCalculator)
    result = unittest.TextTestRunner(verbosity=0).run(suite)

    print("\nOperation          Expected    Actual    Result")
    for operation, expected, actual, status in TestCalculator.results:
```

```
    print(f"{operation:<18}{expected:<11}{actual:<10}{status}")

print("\n----------------------------------------------")
print(f"Ran {result.testsRun} tests")
print("\nOK" if result.wasSuccessful() else "\nFAILED")
```

Test Cases:

Test Case ID	Test Case	Input	Expected Result	Actual Result	Status
TC01	Addition	50 + 29	79	79	PASS
TC02	Subtraction	37 - 7	30	30	PASS
TC03	Multiplication	7 × 49	343	343	PASS
TC04	Division	90 ÷ 5	30	30	PASS

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Desktop\Automation\test_calculator.py"
----------------------------------------------------------------------

Operation          Expected    Actual    Result
Addition           79          79        PASS
Division           30          30.0      PASS
Multiplication     343         343       PASS
Subtraction        30          30        PASS

----------------------------------------------
Ran 4 tests
```

Aim/Overview of the practical (B). Analyze the test cases written in Part 1 and extend them to include Evaluate how unit testing frameworks handle failures and assertions.

Objective: The objective of this experiment is to analyze the test cases developed in Part 1 and extend them to test different input conditions, including valid, invalid, boundary, and exceptional cases. It also aims to evaluate how a unit testing framework handles test failures, assertions, and exceptions, ensuring that errors are properly detected and reported during testing.

Procedure/Algorithm/Code

```
import unittest
results = []

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return float("inf")
    result = a / b
    return int(result) if result.is_integer() else result
class TestCalculator(unittest.TestCase):

    def check(self, operation, equation, actual, expected):
        status = "PASS" if actual == expected else "FAIL"
        results.append([operation, equation, expected, actual, status])
        self.assertEqual(actual, expected)

    def test_boundary_maximum(self):
        self.check("Boundary Value", "1000 - 999",
                   subtract(1000, 999), 1)
    def test_boundary_minimum(self):
        self.check("Boundary Value", "-1000 + 999",
```

```
            add(-1000, 999), -1)

    def test_decimal(self):
        self.check("Decimal Value", "55.7 + 98.7",
                   add(55.7, 98.7), 154.4)

    def test_exception(self):
        try:
            add(30, "D")
            actual = "No Exception"
        except TypeError:
            actual = "Exception"
        expected = "Exception"
        status = "PASS" if actual == expected else "FAIL"
        results.append([
            "Exception Handling",
            "30 + D",
            expected,
            actual,
            status
        ])
        self.assertEqual(actual, expected)

    def test_negative(self):
        self.check("Negative Value", "-27 * 4",
                   multiply(-27, 4), -108)

    def test_zero(self):
        self.check("Zero Value", "56 * 0",
                   multiply(56, 0), 0)

    def test_zero_division(self):
```

```
        actual = divide(40, 0)

        actual = "Infinity" if actual == float("inf") else "Error"
        expected = "Error / Infinity"
        status = "PASS" if actual in ["Error", "Infinity"] else "FAIL"
        results.append([
            "Zero Division",
            "40 / 0",
            expected,
            actual,
            status
        ])
        self.assertIn(actual, ["Error", "Infinity"])
if __name__ == "__main__":

    suite = unittest.defaultTestLoader.loadTestsFromTestCase(
        TestCalculator
    )
    runner = unittest.TextTestRunner(
        verbosity=0,
        stream=open("nul", "w")
    )
    result = runner.run(suite)

    print("\nOperation        Equation      Expected        Actual      Result")
    for r in results:
        print( f"{r[0]:<19} f"{r[1]:<14}"f"{str(r[2]):<18}"  f"{str(r[3]):<13}"f"{r[4]} )
    print("-" * 70)
    print(f"Ran {result.testsRun} tests")
    print("\nOK" if result.wasSuccessful() else "\nFAILED")
```

Test Cases

Test Case ID	Test Case	Test Data	Operation	Expected Result	Actual Result	Status
TC01	Boundary Maximum	1000, 999	1000 - 999	1	0	PASS
TC02	Boundary Minimum	-1000, 999	-1000 + 999	-1	-1	PASS
TC03	Decimal Value	55.7, 98.7	55.7 + 98.7	154.4	154.4	PASS
TC04	Exception Handling	30, "D"	30 + D	Exception	Exception	PASS
TC05	Negative Value	-27, 4	-108	-108	-108	PASS
TC06	Zero Multiply	56, 0	56 × 0	0	0	PASS
TC07	Zero Division	40, 0	40 ÷ 0	Error / Infinity	Infinity	PASS

```
[Running] python -u "c:\Users\Lenovo\OneDrive\Desktop\Automation\Extended_Test_Cases.py"

Operation          Equation      Expected          Actual       Result
Boundary Value     1000 - 999    1                 1            PASS
Boundary Value     -1000 + 999   -1                -1           PASS
Decimal Value      55.7 + 98.7   154.4             154.4        PASS
Exception Handling 30 + D        Exception         Exception    PASS
Negative Value     -27 * 4       -108              -108         PASS
Zero Value         56 * 0        0                 0            PASS
Zero Division      40 / 0        Error / Infinity  Infinity     PASS
.......
----------------------------------------------------------------------
Ran 7 tests
```

Learning outcomes (What I have learnt):

1. Understood the concept and importance of unit testing.
2. Executed test cases using the Python unittest framework.
3. Performed boundary value, decimal, negative, and zero-value testing.
4. Test special cases such as division by zero.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Demonstration and Performance		12
2.	Worksheet		8
3.	Viva		10

Sukhleen Kaur (25MCA20031)
25MCA-6 (B)